

第一次习题课 (hw3)

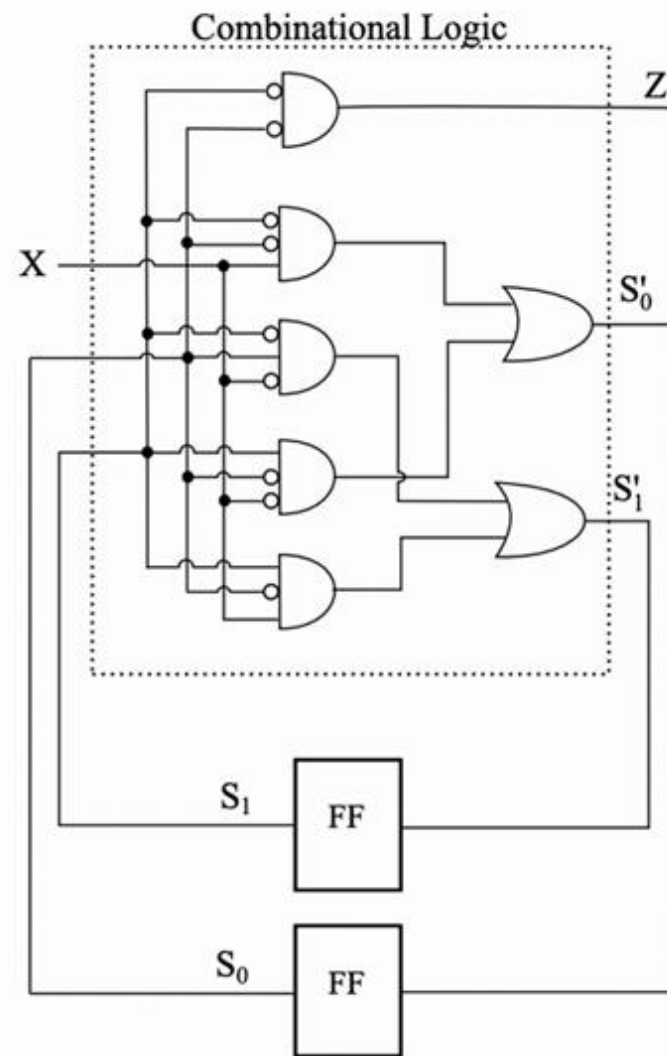
by 殷一鸣

T1

(1) 看图写真值表 (看清线就行)

(2) 根据 (1) 的真值表画状态机 (逐行画, 注意输出的 s_0' , s_1' 为下一次输入的 s_0, s_1)

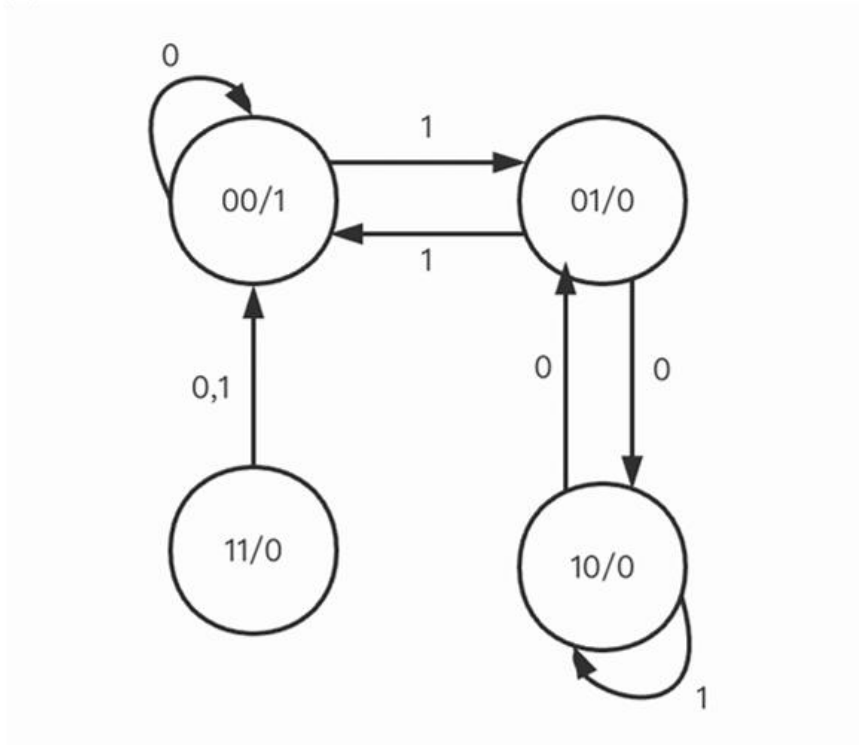
The logic diagram shown below is a finite state machine.



(1).

S_1	S_0	X	Z	S'_1	S'_0
0	0	0	1	0	0
0	0	1	1	0	1
0	1	0	0	1	0
0	1	1	0	0	0
1	0	0	0	0	1
1	0	1	0	1	0
1	1	0	0	0	0
1	1	1	0	0	0

(2).



T2

```
AND R0,R0,0
ADD R0,R0,0x10(2)
if(>=0)BR PCoffset9 (注意此时PC已是x3003)
ADD R0,R0, 0X11(3)
ADD R0,R0, 0X1(1)
TRAP 0010 0101
```

前 4 个都比较显然,查书即可,第 5 个注意循环 R0 循环加 2 直到加到 x8000后循环中止,再加上后面的 3 和 1,得到 x8004.

X	Does the program halt?	Value stored in R0
000000010	Yes	2
000000001	Yes	3
000000000	Yes	6
111111111	No	--
111111110	Yes	x8004 ($-2^{15} + 4$)

一个LC-3程序存储在内存地址x3000到x3005中。注意，内存地址x3002中的分支指令有一个未指定的PCoffset9，记为X。

Address	Instruction
x3000	0101 000 000 1 00000
x3001	0001 000 000 1 00010
x3002	0000 011 X
x3003	0001 000 000 1 00011
x3004	0001 000 000 1 00001
x3005	1111 0000 0010 0101

程序从PC = x4000开始执行。

你的任务：在下表中，对于X的每个值，回答问题："程序是否会停止？"（是或否）。如果你的答案是"是"，回答问题："在x4004处的指令执行完成后，R0中立即存储的值是多少？"如果你的答案是"否"，在"R0中存储的值"列中填入一个破折号。

X	Does the program halt?	Value stored in R0
000000010		
000000001		
000000000		
111111111		
111111110		

T3

Instruction Processing: **FETCH**

■ Load next instruction (at address stored in PC) from memory into Instruction Register (IR).

- Load contents of PC into MAR.
- Send "read" signal to memory.
- Read contents of MDR, store in IR.

■ Then increment PC, so that it points to the next instruction in sequence.

- PC becomes PC+1.

假设从内存读取或写入需要100个周期，而从寄存器读取或写入只需要一个周期。

计算LC-3指令的每个指令周期阶段所需的周期数

1 | ADD R6, R2, R6

假设每个阶段（如果需要）需要一个周期，除非需要访问内存。

FETCH: $1 + 100 + 1 = 102$

DECODE: 1 (memory access is not required)

EVALUATE ADDRESS: 0 (this phase is not required)

FETCH OPERANDS: 1 (memory access is not required)

EXECUTE: 1 (memory access is not required)

STORE RESULT: 0 (this phase is not required)

T4

假设我们将LC-3改为只有4个寄存器而不是8个。更少的寄存器通常是一个坏主意，因为这意味着需要更频繁地从内存加载和存储到内存，但我们仍然可以问这个问题：减少寄存器数量会有什么好处吗？对于以下每个问题，回答是或否，并解释你的答案。

1. 如果我们保持所有指令的基本格式不变（并保持每条指令为16位），那么如果我们将寄存器数量减少到4个，对于操作指令（0001, 0101, 1001）是否有任何好处？
2. 如果我们将寄存器数量减少到4个，对于加载（0010）和存储（0011）指令是否有任何好处？
3. 如果我们将寄存器数量减少到4个，对于条件分支（0000）指令是否有任何好处？

1. ADD(0001) 和 AND(0101) 有了更大的立即数范围。但 NOT(1001) 没有获益。
2. LD(0010) 和 ST(0011) 能有更多的一位去寻址。
3. BR(0000) 中没有寄存器，因此没有获益。

T5

- a. OPCODE 有 225 条，需要 8 位才能表示完整
- b. 寄存器有 120 个，需要 7 位才能表示完整
- c. $32 - 7 * 3 - 8 = 3$ ，最多有 3 位 UNUSED

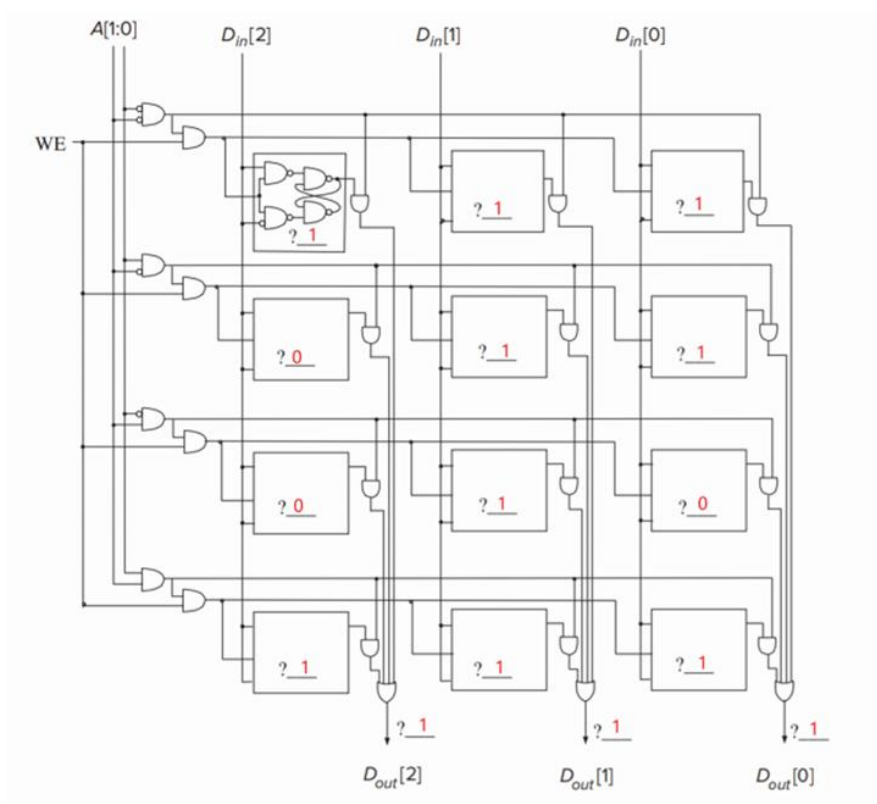
假设一条32位指令采用以下格式：

OPCODE	DR	SR1	SR2	UNUSED

如果有225个操作码和120个寄存器，

- (a). 表示操作码（OPCODE）所需的最小位数是多少？
- (b). 表示目标寄存器（DR）所需的最小位数是多少？
- (c). 指令编码中未使用位的最大数量是多少？

T6



只要看每个地址最后一次被写时Din的值即可，
则地址 00 应该是 cycle 6 写的 111, 01 是 cycle 4 写的 011, 10 是 cycle 3 写的 010, 11 是 cycle 7 写的 111, 看最后一个 cycle, 在读 11 处的值, 所以读出 111.

课上讨论的 2^2 乘3位内存在连续五个时钟周期内被访问。下表显示了每次访问期间两位地址、一位写使能和三位数据输入信号的值。

	A[1:0]	WE	$D_{in}[2:0]$
cycle 1	0 1	1	1 0 1
cycle 2	1 1	0	1 1 0
cycle 3	<u>1 0</u>	1	0 1 0
cycle 4	<u>0 1</u>	1	0 1 1
cycle 5	1 1	0	1 0 1
cycle 6	<u>0 0</u>	1	1 1 1
cycle 7	<u>1 1</u>	<u>1</u>	1 1 1
cycle 8	1 1	<u>0</u>	0 1 0

你的任务：填写第八个周期结束前每个内存单元中存储的值和三条数据输出线的值。假设最初所有12个内存单元都存储值1。在下图中，每个问号（?）表示你需要填写的值。

T8

以下用 $a \oplus b$ 表示 a XOR b, $a + b$ 表示 a OR b, $a \cdot b$ 表示 a AND b, $\bar{a}$ 表示 NOT a, 则有 $a \oplus b = a \cdot \bar{b} + \bar{a} \cdot b = \overline{a \cdot \bar{b} + \bar{a} \cdot b} = \overline{\bar{a} \cdot b} \cdot \overline{a \cdot \bar{b}}$.

Address	Instruction	Comments
x3000	1001 111 001 111111	NOT R7, R1
x3001	1001 110 010 111111	NOT R6, R2
x3002	0101 101 111 000 010	AND R5, R7, R2
x3003	0101 100 110 000 001	AND R4, R6, R1
x3004	1001 001 101 111111	NOT R1, R5
x3005	1001 010 100 111111	NOT R2, R4
x3006	0101 000 001 000 010	AND R0, R1, R2
x3007	1001 011 000 111111	NOT R3, R0

LC-3没有用于逻辑异或（XOR）功能的操作码。下面的八条指令序列对R1和R2的内容执行异或操作，并将结果放入R3。填写四条缺失的指令，使这八条指令序列能够完成任务。

Address	Instruction
x3000	
x3001	1001 110 010 111111
x3002	0101 101 111 000 010
x3003	
x3004	1001 001 101 111111
x3005	
x3006	
x3007	1001 011 000 111111

T9

我们希望有一条什么都不做的指令。许多ISA实际上都有一个专门用于什么都不做的操作码。它通常被称为NOP，意思是无操作。该指令被取出、解码和执行。执行阶段就是什么都不做！以下五条指令中哪些可以用作NOP并使程序仍然正常工作？对于其他指令，请描述它们做了什么。

1. 0001 010 001 1 00010
2. 0000 111 000000000
3. 0000 101 000000100
4. 1001 010 111 111111
5. 1111 0000 00100011

其中第2条可以作为NOP，因为其偏移设置为0

1. 将R1中的值与立即数2相加并存放到R2中
2. NOP
3. 检查标志位N和P，若其中一个被置位则跳转到对应的位置
4. 对R7求反存放到R2中
5. TRAP指令，读取内存单元x0023中的内容作为服务程序入口

T10

请描述LC-3中BR指令的限制以及JMP指令如何解决这个问题（请给出一个简单的例子）

BR指令无法跳转到PC寄存器和偏移量之和范围以外的指令，即其最大范围为距离当前指令+256到-255的地址空间

对于JMP指令，其能够将对应寄存器的内容装入PC寄存器，使得程序执行流可以跳转至内存空间的任意位置

T7 *一定要会*

Operation No.	R/W	MAR	MDR
1	W	x <u>4</u> 000	11110
2	R	x <u>4</u> 003	<u>1</u> 0110
3	W	x <u>4</u> 001	10 <u>1</u> 10
4	R	x <u>4</u> 002	<u>0</u> 1101
5	<u>W</u>	x <u>4</u> 003	<u>0</u> 1101

Operations on Memory

下表显示了在第一次访问之前、第三次访问之后和第五次访问之后，内存地址x4000到x4004的内容。我们为这个问题添加了一个约束条件以获得一个正确的答案：MDR只能作为加载（读取）访问的结果从内存加载。

Address	Before Access 1	After Access 3	After Access 5
x4000	01101	___0 11110	___ 11110
x4001	11010	_0_0 10110	___ 10110
x4002	_1___ 01101	___ 01101	___ 01101
x4003	10110	___ 10110	01101
x4004	11110	11110	11110

Contents of Memory locations

You're required to fill in the blanks.

为了叙述方便, 下面会简称 Operation i 为 i, 表 1 指代 Operations on Memory, 表 2, 3, 4 指代 Memory before Access 1, Memory after Access 3, Memory after Access 5. 同时建议大家在看的同时按照我的推导填表. 因为The data in the MDR must come from a previous read (load). 而 Operation 3 的 MDR 与 1 有不同, 说明 Operation 2 是 Read. 而 Memory 在 Access 3 后与 Access 1 前 x4000 和 x4001 都发生了变化, 说明 Operation 1 和 3 写了这两处地址, 假设 1 写了 x4001, 与第二位是 0 矛盾, 因此 1 写了 x4000, 3 写了 x4001, 同时该步的 MDR 是 10xx0, 比对表 2, 知道应该来自 x4003. 到这里填出了表 1 前 3 行和表2, 3 除了 x4002 外的行. 之后发现 x4003 在 Access 5 由 10110 变成了 01101, 发生了 Write, 说明 4 是 R, 5 是 W, 5 的 MDR 是 01101, 考虑到表 3 在上一步填完后没有 01101, 说明 x4002 处值为 01101, 得解, 最终答案如上。